Grundgebiete der Informatik 1

Programmierung, Algorithmen und Datenstrukturen

Prof. Dr.-Ing. Tobias Gemmeke

Integrierte Digitale Systeme und Schaltungsentwurf (IDS)

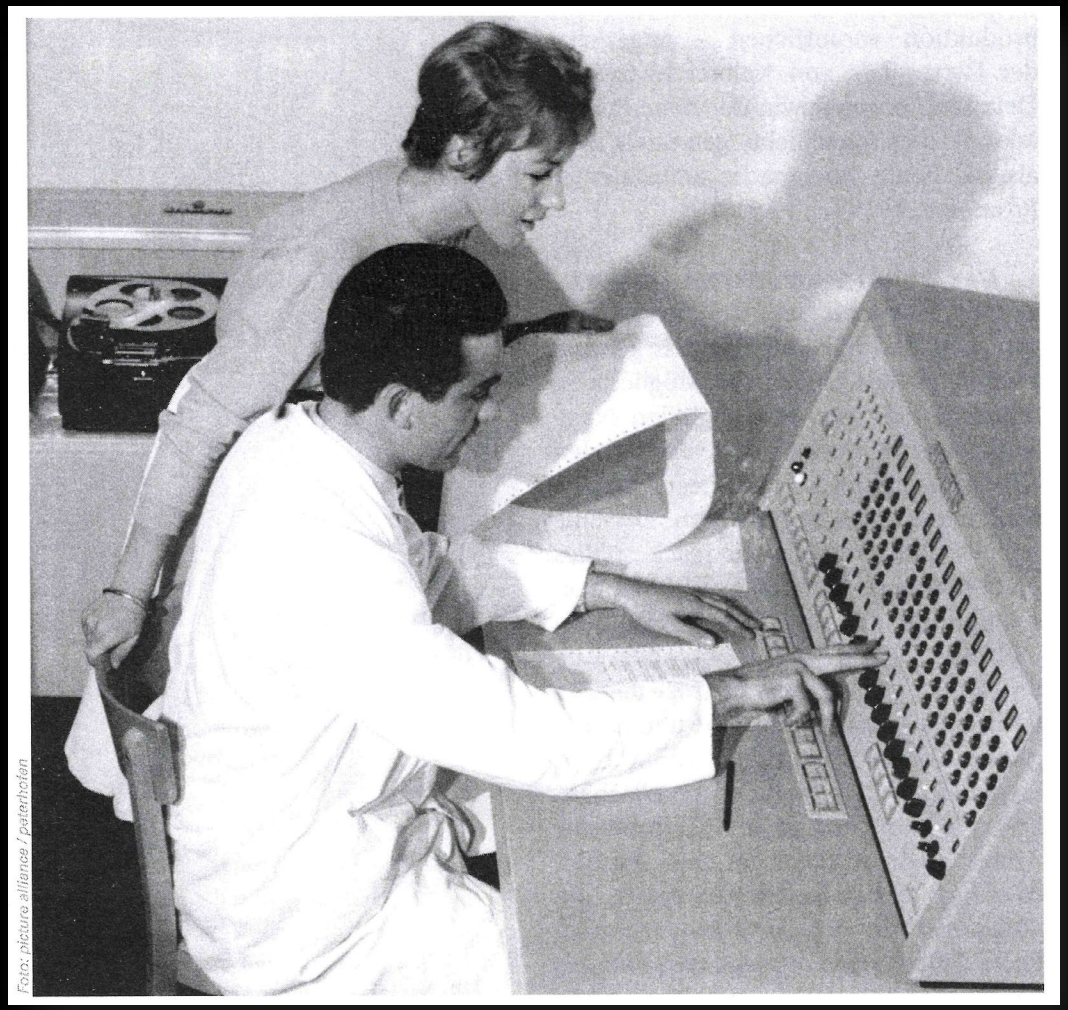
gemmeke@ids.rwth-aachen.de



RWTH AACHEN UNIVERSITY

WS 2026/2027

Computerstudium an der elektronischen Großrechenanlage der TH Aachen um 1960



[Forschung & Lehre, 3/24]

CIP-Pool 2050



www.odditycentral.com



generated with chatGPT



techrepublic.com

CIP-Pool 2026



Praktika
&
Lernraum

Übungsbetrieb

- **Koordination / Fachstudienbetreuung**
  - Dipl.-Inform. Tatjana Eiden
  - Fragestunden
    - Di 12:15-13:45 (UMIC, ab 20.10.)
    - Do 13:15-14:00 (H02, ab 15.10.) im Anschluss an die Globalübung



- **Gemeinsame Globalübung**
  - Do 12:15-13:15
  - Vor allem: c/C++-Programmierkurs
  - Michael Gansen



- **Kleingruppenübungen:**
  - Interaktive Vertiefung und Ergänzung des Vorlesungsstoffes
  - Tutoren: studentische Hilfskräfte
  - Ca. 16 wöchentliche Termine zur Auswahl
  - Vorauss. ab 2. Vorlesungswoche (siehe RWTHonline)

© T. Gemmeke, IDS, RWTH

Kleingruppenübungen

- Teilnahme nicht obligatorisch aber dringend empfohlen
- Anmeldungsinfo: **siehe RWTHonline**
- Anmeldeschluss beachten!
- **Gruppeneinteilung**: Benachrichtigung per email
- **Übungsaufgaben**: jeweils am Fr online über Moodle
- Vorbereitungsaufgaben, Präsenzaufgaben und Nachbereitungsaufgaben → 100P Voraussetzung für Probeklausur

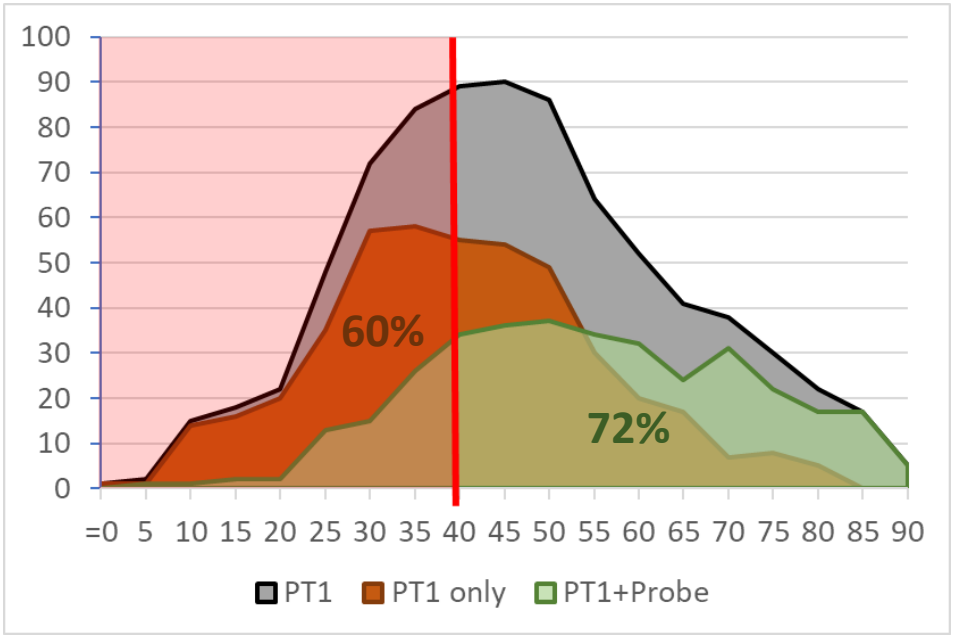
Alle Studierende mit 100 von 200 Punkten in den Hausübungen **haben** in 21/22 **bestanden.**

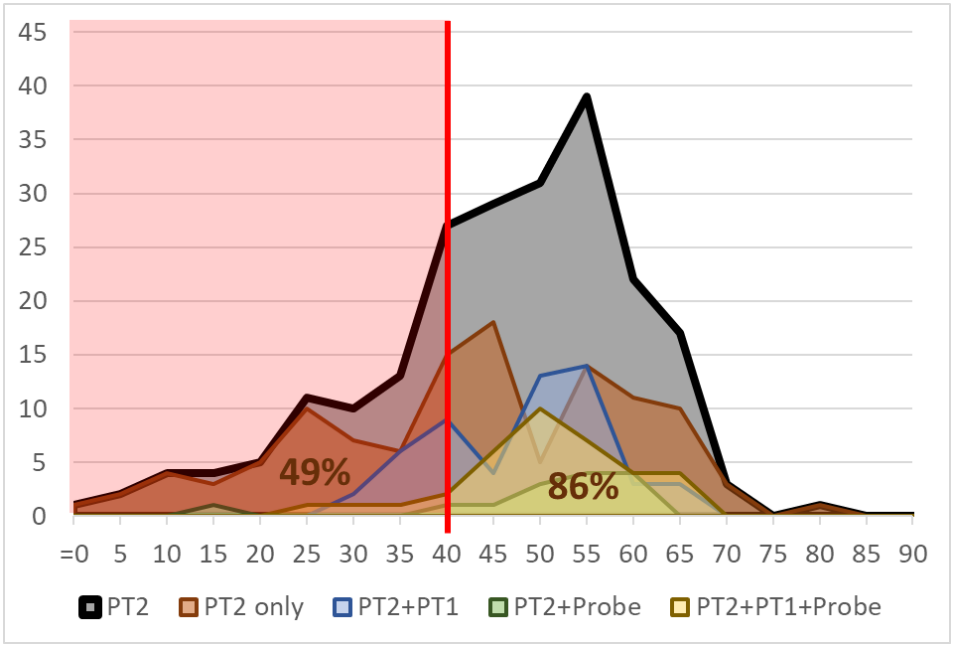
Teilnahme an der **Probeklausur** führte in 21/22 zu folgender **Verbesserung:**

$$\mathbf{2} \xrightarrow{50\%} \mathbf{1} \qquad \mathbf{3} \xrightarrow{90\%} \mathbf{<3} \qquad \mathbf{4} \xrightarrow{90\%} \mathbf{<4} \qquad \mathbf{5} \xrightarrow{80\%} \mathbf{<5}$$

© T. Gemmeke, IDS, RWTH

Klausur



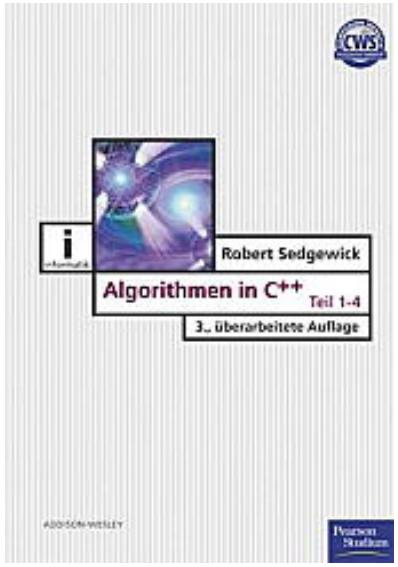


© T. Gemmeke, IDS, RWTH

Literatur

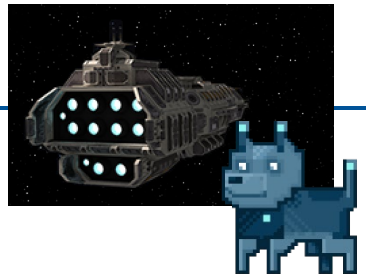
- B.W. Kernighan, D.M. Ritchie:
  Programmieren in C,
  2. Auflage, Hanser, 1990, ISBN 3-446-15497-3
- R. Sedgewick: Algorithmen in C++,
  Addison-Wesley, 2002, ISBN 3-8273-7026-4
- In der Lehrbuchsammlung
- Kommentierte Vorlesungsfolien (elektronisch)
- Grundsätzlich auch andere Einführungen in die Programmierung als Hilfsmittel geeignet



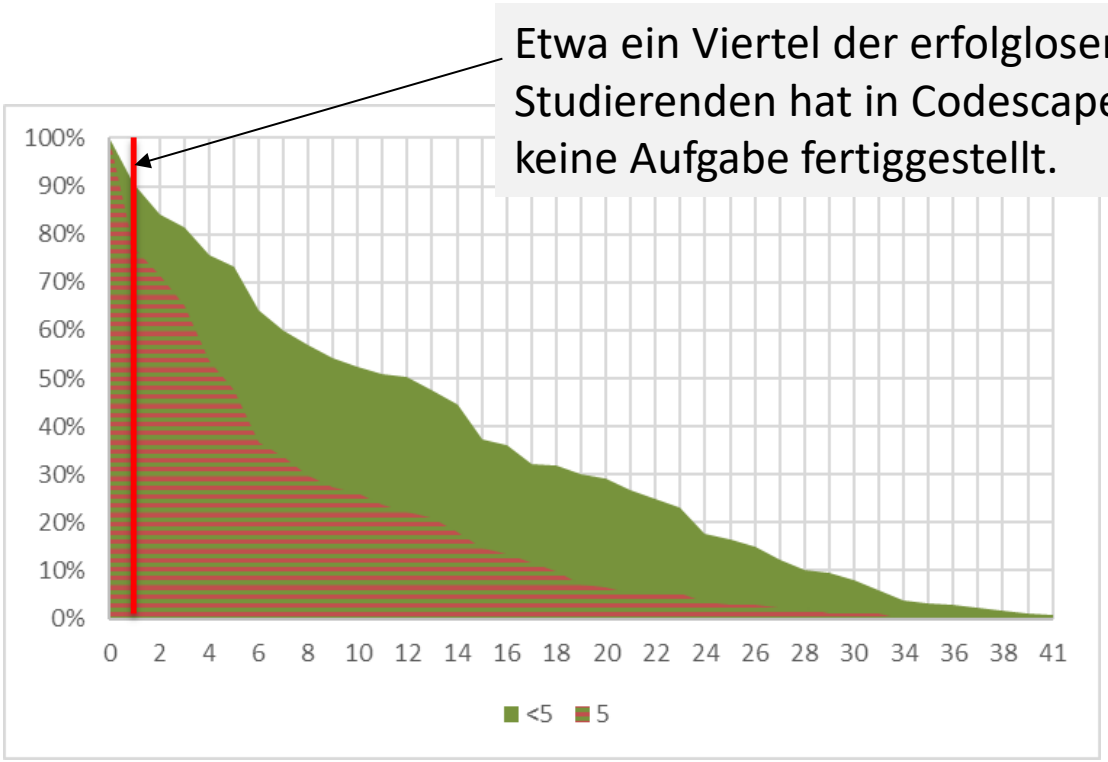


© T. Gemmeke, IDS, RWTH

CODESCAPE



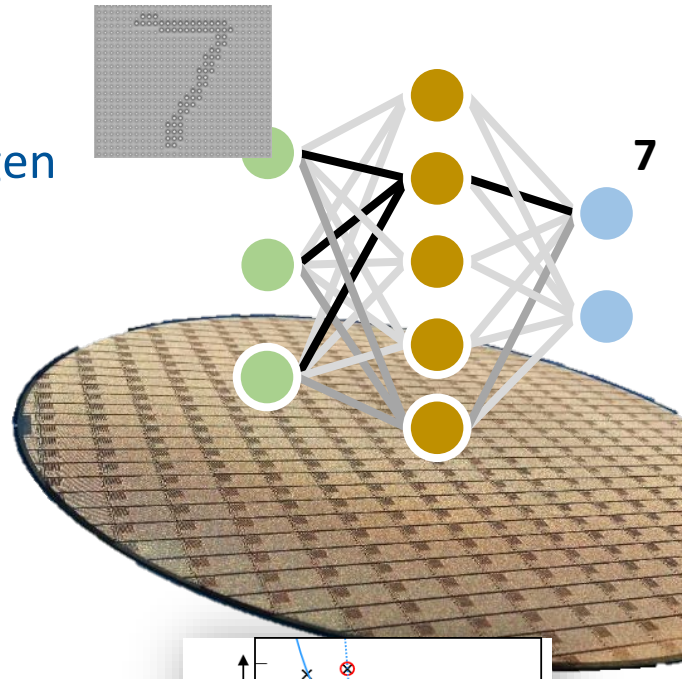
- Serious Gaming
- Grundlagen von Programmiersprachen
- Anwendung von Algorithmen
- Eigene Lösungsideen entwickeln

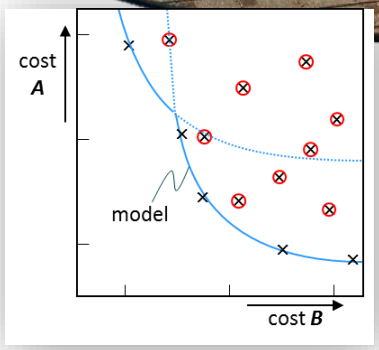


© T. Gemmeke, IDS, RWTH

Über das IDS (www.ids.rwth-aachen.de)

- Lehrstuhl für Integrierte Digitale Systeme und Schaltungsentwurf (IDS)
- Schwerpunkte:
  - Entwurfsmethodik integrierte digitale Schaltungen
  - Beschleuniger für verschiedene Anwendungen:
    - Simulation biologischer neuronaler Netzwerke
    - Hardware für künstliche neuronale Netze
    - Kryptographie
    - …
  - Testmethoden
- Standort: UMIC Gebäude
  - Mies-van-der-Rohe Str. 15, RWTH Hörn Campus



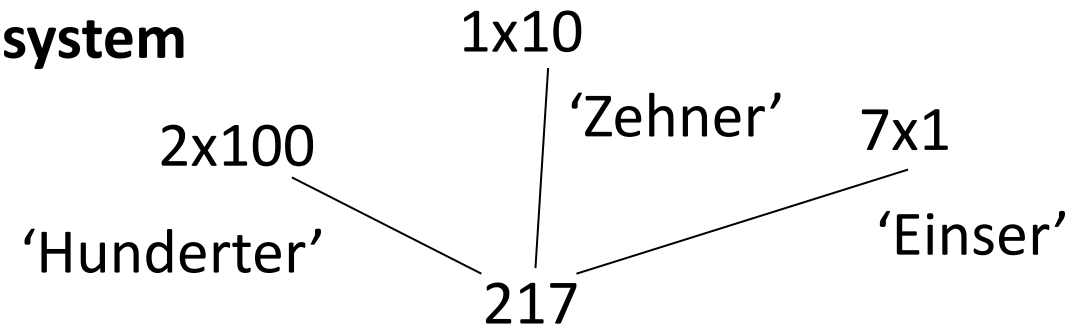




© T. Gemmeke, IDS, RWTH

Zahlendarstellung

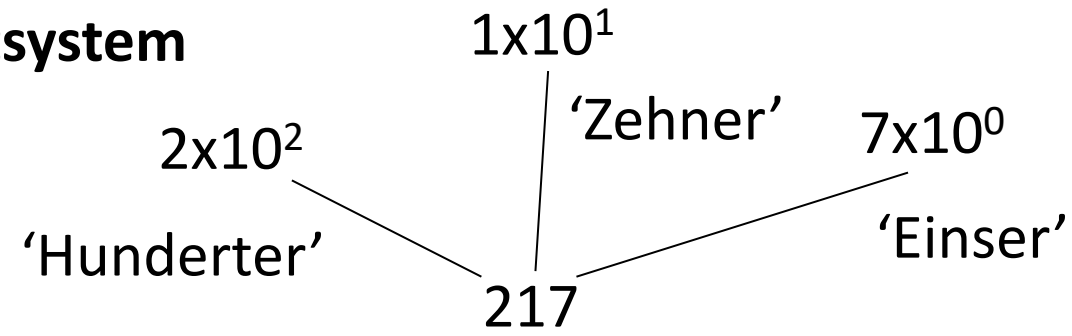
Stellenwertsystem



© T. Gemmeke, IDS, RWTH

Zahlendarstellung

Stellenwertsystem



Dezimalsystem $(r = 10)$

Gewicht	10^2	10^1	10^0
Ziffern / Digits	2	1	7

$$X = \sum_{i=0}^{N-1} x_i \cdot r^i$$

$2x10^2 + 1x10^1 + 7x10^0$

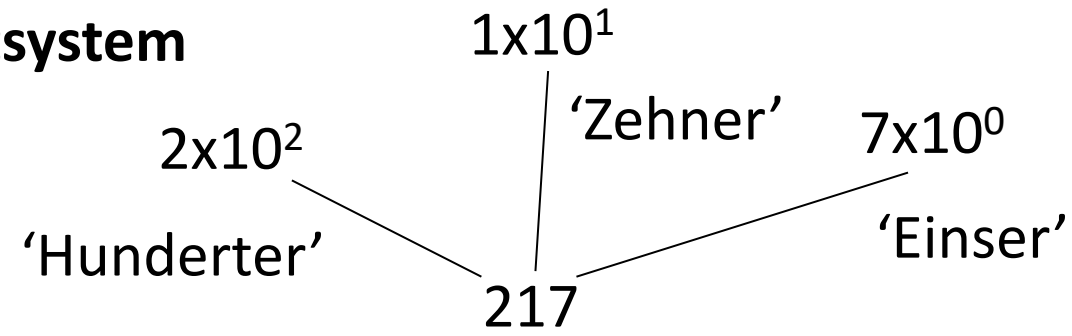
Binärystem $(r = 2)$

Gewicht	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Ziffern / Digits								

© T. Gemmeke, IDS, RWTH

Zahlendarstellung

Stellenwertsystem



$$X = \sum_{i=0}^{N-1} x_i \cdot r^i$$

Dezimalsystem ($r = 10$)

Gewicht	10^2	10^1	10^0
Ziffern / Digits	2	1	7

Binärystem ($r = 2$)

Gewicht	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Ziffern / Digits	1	1	0	1	1	0	0	1

© T. Gemmeke, IDS, RWTH

Zahlendarstellung

$(0010)_2$ $(002)_{10}$

$(0111)_2$ $(007)_{10}$

$(????)_2$ $(016)_{10}$

$(0010)_{16}$

$(1101\ 1001)_2$ $(217)_{10}$

$(13)_{10}=(?)_{16}$

$(\boxed{\ }9)_{16}$

© T. Gemmeke, IDS, RWTH

Zahlendarstellung

Dezimalsystem ($r = 10$)

Gewicht	10^2	10^1	10^0
Ziffern / Digits	2	1	7

$$X = \sum_{i=0}^{N-1} x_i \cdot r^i$$

Binärystem ($r = 2$)

Gewicht	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Ziffern / Digits	1	1	0	1	1	0	0	1

Hexadezimalystem ($r = 16$)

Gewicht	16^7	16^6	16^5	16^4	16^3	16^2	16^1	16^0
Ziffern / Digits	0	0	0	0	0	0		9

[playing card: Wikipedia]

© T. Gemmeke, IDS, RWTH

Zahlendarstellung

r	10	2	16
	00	0000 0000	00
	01	0000 0001	01
	02	0000 0010	02
	03	0000 0011	03
	04	0000 0100	04
	05	0000 0101	05
	06	0000 0110	06
	07	0000 0111	07
	08	0000 1000	08
	09	0000 1001	09
	10	0000 1010	
	11	0000 1011	
	12	0000 1100	
	13	0000 1101	
	14	0000 1110	
	15	0000 1111	
	16	0001 0000	
	17	0001 0001	



?

© T. Gemmeke, IDS, RWTH

Zahlendarstellung

r	10	2	16
	00	0000 0000	00
	01	0000 0001	01
	02	0000 0010	02
	03	0000 0011	03
	04	0000 0100	04
	05	0000 0101	05
	06	0000 0110	06
	07	0000 0111	07
	08	0000 1000	08
	09	0000 1001	09
	10	0000 1010	0A
	11	0000 1011	0B
	12	0000 1100	0C
	13	0000 1101	0D
	14	0000 1110	0E
	15	0000 1111	0F
	16	0001 0000	
	17	0001 0001	

© T. Gemmeke, IDS, RWTH

Zahlendarstellung

r	10	2	16
	00	0000 0000	00
	01	0000 0001	01
	02	0000 0010	02
	03	0000 0011	03
	04	0000 0100	04
	05	0000 0101	05
	06	0000 0110	06
	07	0000 0111	07
	08	0000 1000	08
	09	0000 1001	09
	10	0000 1010	0A
	11	0000 1011	0B
	12	0000 1100	0C
	13	0000 1101	0D
	14	0000 1110	0E
	15	0000 1111	0F
	16	0001 0000	10
	17	0001 0001	11

© T. Gemmeke, IDS, RWTH

Buchstabendarstellung

American Standard Code for Information Interchange (ASCII)

USASCII code chart

Bits b_4	b_3	b_2	b_1	Row ↓ / Column →	0 (000)	1 (001)	2 (010)	3 (011)	4 (100)	5 (101)	6 (110)	7 (111)
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	|
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

[General Electric TermiNet 300, ~1971]

```
char c0 = 0x61;
char c1 = 'a';
printf("char c0:%c  char c1:%c\n", c0, c1);
```

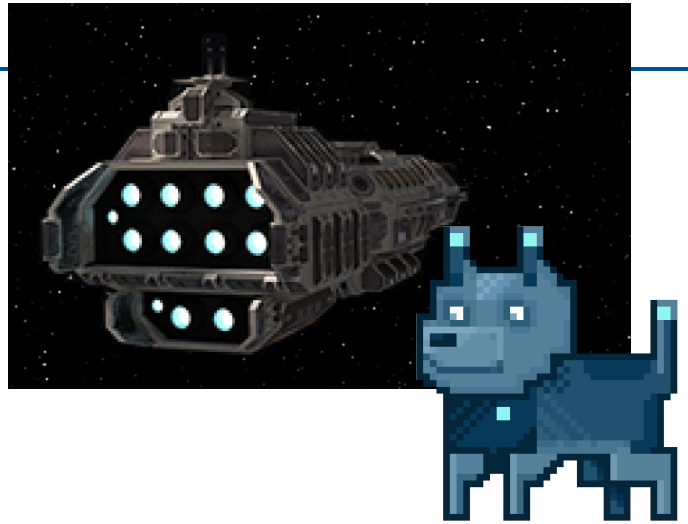
```
> char c0:a  char c1:a
```



Unicode Character
UTF 0x0001F44D

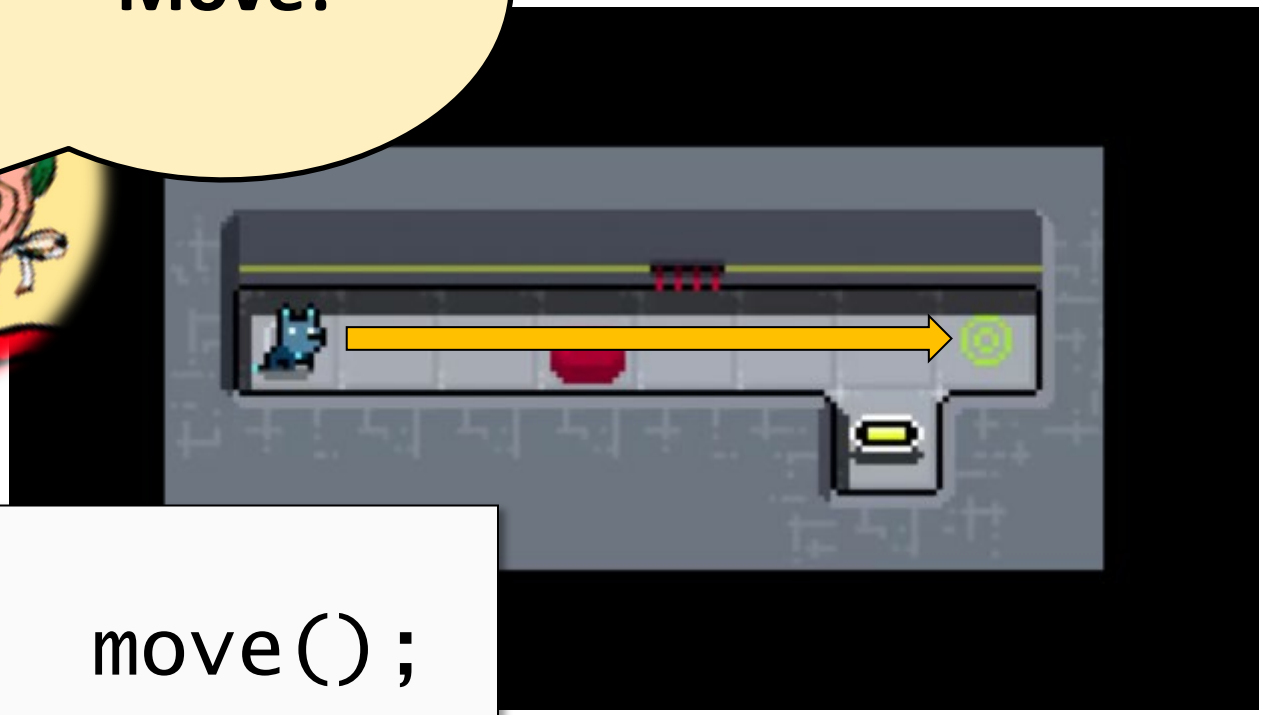
© T. Gemmeke, IDS, RWTH

CODESCAPE



- Serious Gaming
- Grundlagen von Programmiersprachen
- Anwendung [illegible]orithmen
- Eigene [illegible]ickeln



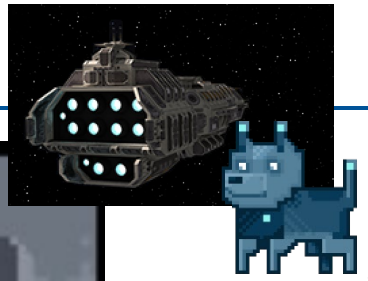


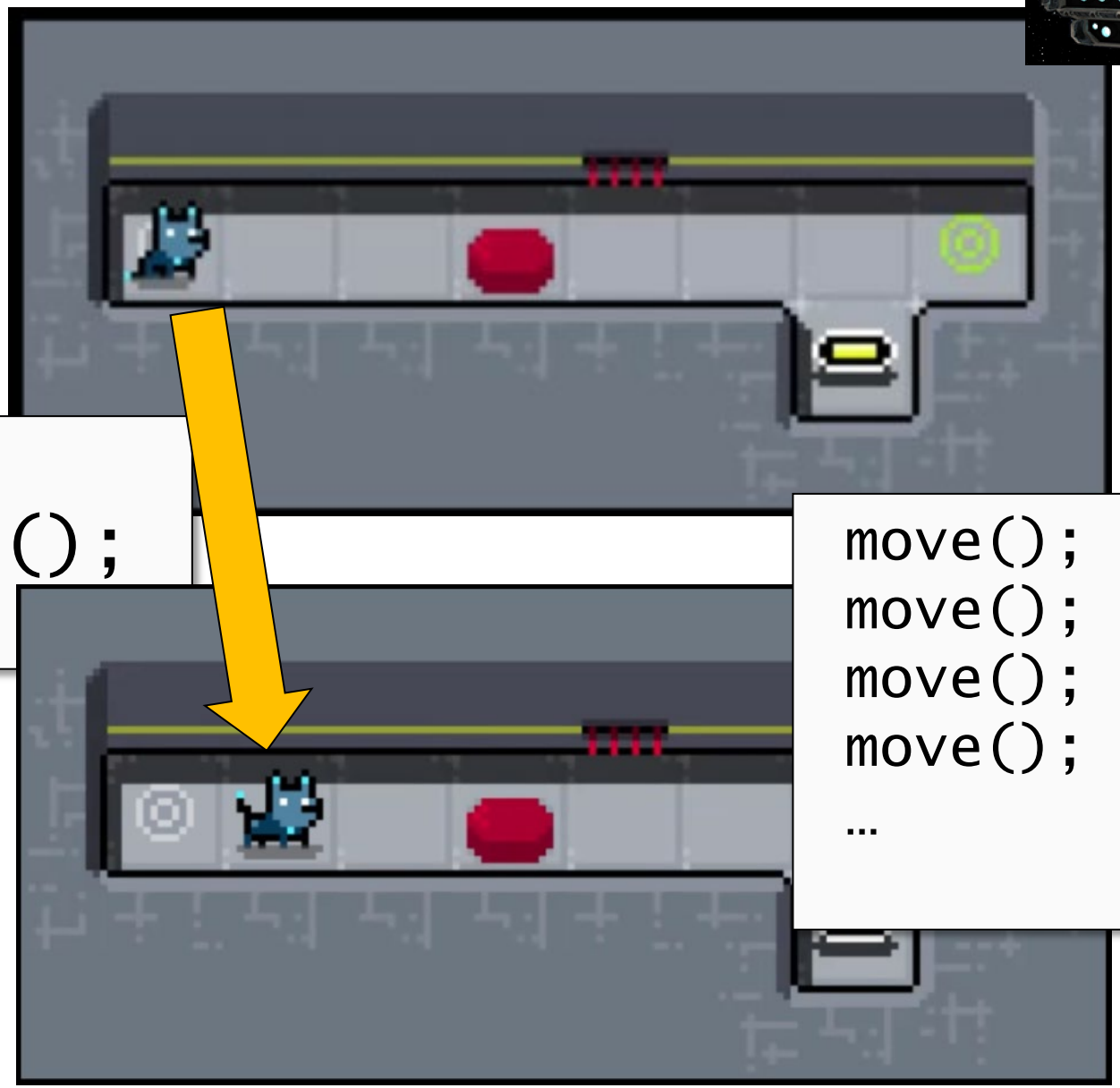
```
move();
```

 [casesar: asterix.com]

© T. Gemmeke, IDS, RWTH

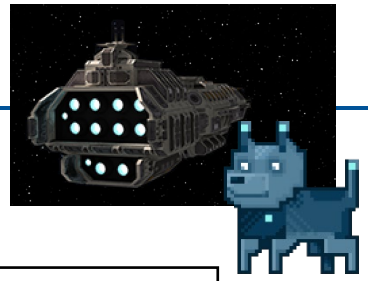
CODESCAPE





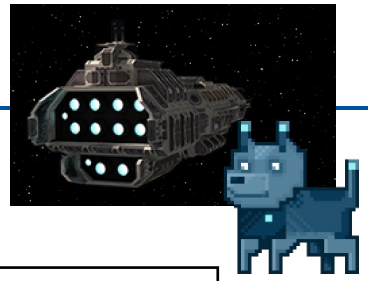
© T. Gemmeke, IDS, RWTH

CODESCAPE



© T. Gemmeke, IDS, RWTH

CODESCAPE



© T. Gemmeke, IDS, RWTH

Inhalt der Vorlesung

1. Einleitung
2. Einführendes Beispiel
3. Elementare Datentypen und Anweisungen
4. Zusammengesetzte Datentypen
5. Funktionen und Programmaufbau
6. Analyse von Algorithmen
7. Sortieralgorithmen
8. Lineare Datentypen
9. Bäume
10. Graphen
11. Techniken zum Algorithmenentwurf

© T. Gemmeke, IDS, RWTH

2. Einführendes Beispiel

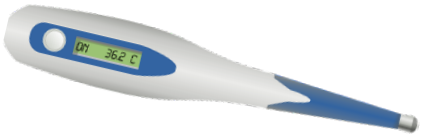
2.1 Temperaturumrechnung

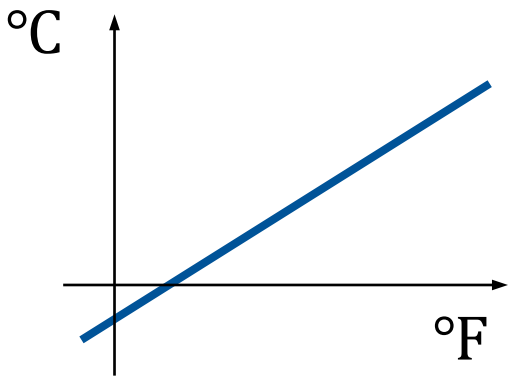
2.2 Analyse des Beispiels

2.3 Ausführung und Fehlersuche

Einführendes C-Programmbeispiel

- Umwandlung von Grad Fahrenheit in Grad Celsius
- $\frac{y}{°C} = (5/9) \times \left(\frac{x}{°F} - 32\right)$





© T. Gemmeke, IDS, RWTH



Sven Moschitz, Bild, 2.5.2024



@eomhyeok, unsplash, 30.11.2022

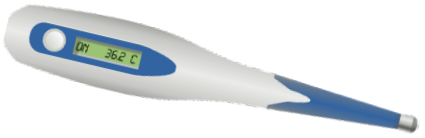


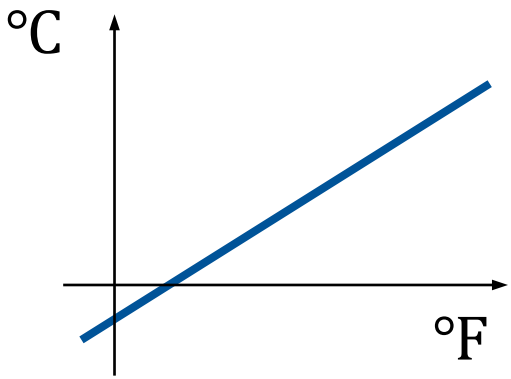
Joshua Mellin, CNN, 17.12.2018

Einführendes C-Programmbeispiel

- Umwandlung von Grad Fahrenheit in Grad Celsius
- $\frac{y}{°C} = (5/9) \times \left(\frac{x}{°F} - 32\right)$

```
/* Umwandlung von Fahrenheit in Celsius
    fuer fahr = 0, 20, …, 300   */
```





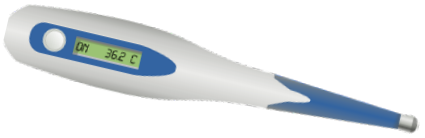
© T. Gemmeke, IDS, RWTH

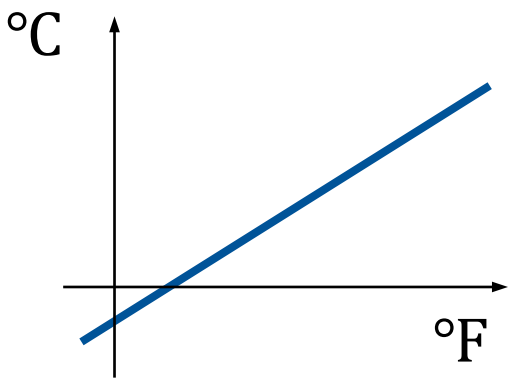
Einführendes C-Programmbeispiel

- Umwandlung von Grad Fahrenheit in Grad Celsius
- $\frac{y}{°C} = (5/9) \times \left(\frac{x}{°F} - 32\right)$

```
/* Umwandlung von Fahrenheit in Celsius
    fuer fahr = 0, 20, …, 300   */

        celsius = 5  *  (fahr-32) / 9;
```





© T. Gemmeke, IDS, RWTH

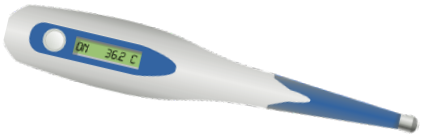
Einführendes C-Programmbeispiel

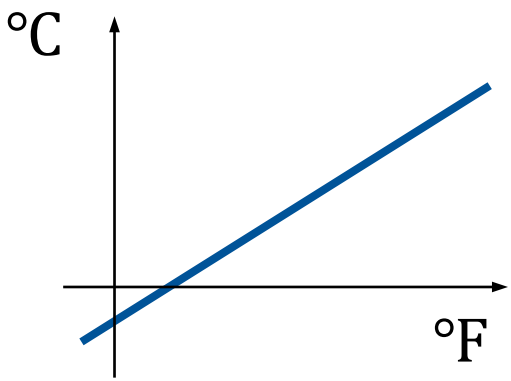
- Umwandlung von Grad Fahrenheit in Grad Celsius
- $\frac{y}{°C} = (5/9) \times \left(\frac{x}{°F} - 32\right)$

```
/* Umwandlung von Fahrenheit in Celsius
    fuer fahr = 0, 20, …, 300   */

    int fahr

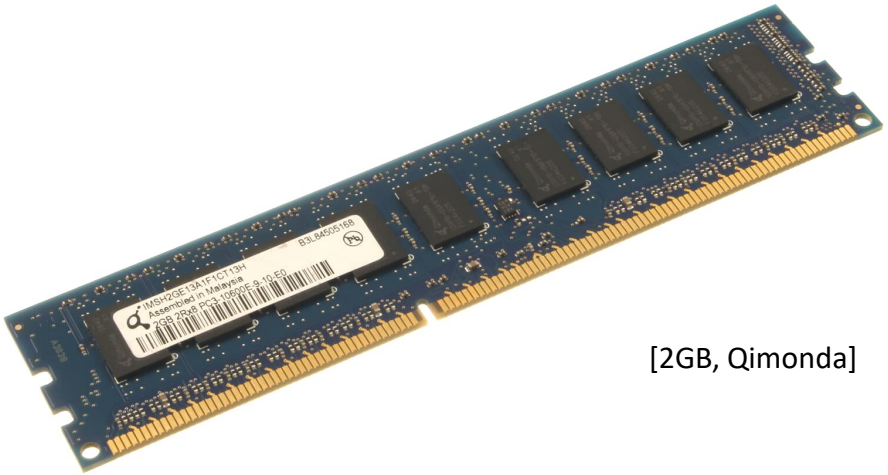
        celsius = 5  *  (fahr-32) / 9;
```





© T. Gemmeke, IDS, RWTH

Speicher



[2GB, Qimonda]



[Schubladenschrank “Alex”, ikea]

© T. Gemmeke, IDS, RWTH

Einführendes C-Programmbeispiel

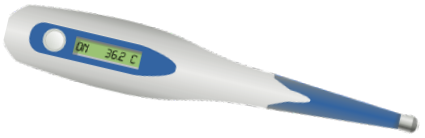
- Umwandlung von Grad Fahrenheit in Grad Celsius
- $\frac{y}{°C} = (5/9) \times \left(\frac{x}{°F} - 32\right)$

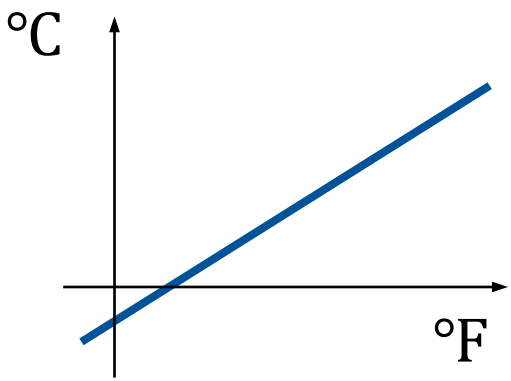
```
/* Umwandlung von Fahrenheit in Celsius
    fuer fahr = 0, 20, …, 300   */

    int fahr




        celsius = 5  *  (fahr-32) / 9;
```





© T. Gemmeke, IDS, RWTH

Einführendes C-Programmbeispiel

- Umwandlung von Grad Fahrenheit in Grad Celsius
- $\frac{y}{°C} = (5/9) \times \left(\frac{x}{°F} - 32\right)$

```c
/* Umwandlung von Fahrenheit in Celsius
    fuer fahr = 0, 20, …, 300   */

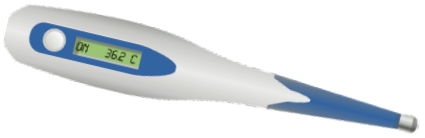

    int fahr, celsius;
    int lower

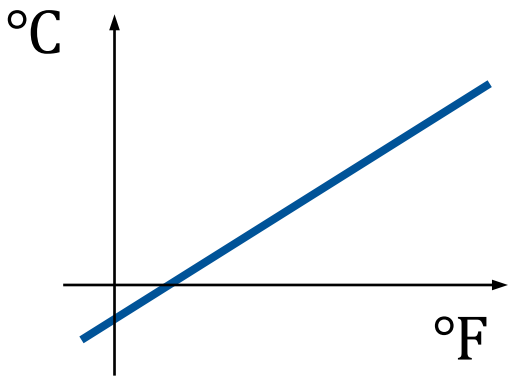
    lower = 0; // untere Grenze der Temperaturtabelle



    fahr  = lower;

        celsius = 5  *  (fahr-32) / 9;
```





© T. Gemmeke, IDS, RWTH

Einführendes C-Programmbeispiel

- Umwandlung von Grad Fahrenheit in Grad Celsius
- $\frac{y}{°C} = (5/9) \times \left(\frac{x}{°F} - 32\right)$

```c
/* Umwandlung von Fahrenheit in Celsius
    fuer fahr = 0, 20, …, 300   */

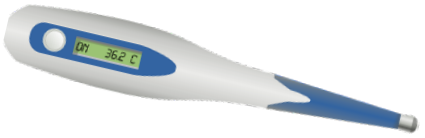

    int fahr, celsius;
    int lower,

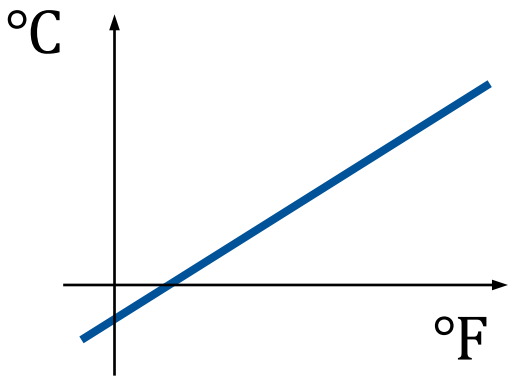
    lower = 0; // untere Grenze der Temperaturtabelle



    fahr  = lower;
    while (fahr <= upper)  {
        celsius = 5  *  (fahr-32) / 9;

        fahr = fahr + step;
    }
```





© T. Gemmeke, IDS, RWTH

Einführendes C-Programmbeispiel

- Umwandlung von Grad Fahrenheit in Grad Celsius
- $\frac{y}{°C} = (5/9) \times \left(\frac{x}{°F} - 32\right)$

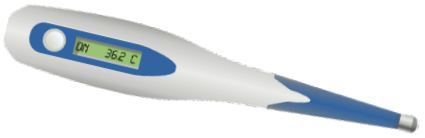
```
/* Umwandlung von Fahrenheit in Celsius
    fuer fahr = 0, 20, …, 300   */

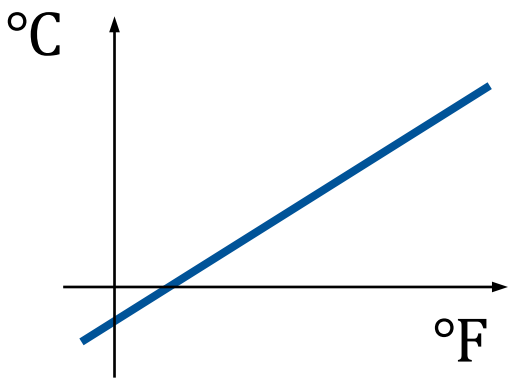

    int fahr, celsius;
    int lower

    lower = 0; // untere Grenze der Temperaturtabelle



    fahr  = lower;
    while (fahr <= upper)  {
        celsius = 5  *  (fahr-32) / 9;
        printf("%d\t%d\n", fahr, celsius);
        fahr = fahr + step;
    }
```





© T. Gemmeke, IDS, RWTH

American Standard Code for Information Interchange (ASCII)

USASCII code chart

Bits b_7 b_6 b_5 →	0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1
b_4 b_3 b_2 b_1 ↓ Row / Column →	0	1	2	3	4	5	6	7
0 0 0 0	0	NUL	DLE	SP	0	@	P	`	p
0 0 0 1	1	SOH	DC1	!	1	A	Q	a	q
0 0 1 0	2	STX	DC2	"	2	B	R	b	r
0 0 1 1	3	ETX	DC3	#	3	C	S	c	s
0 1 0 0	4	EOT	DC4	$	4	D	T	d	t
0 1 0 1	5	ENQ	NAK	%	5	E	U	e	u
0 1 1 0	6	ACK	SYN	&	6	F	V	f	v
0 1 1 1	7	BEL	ETB	'	7	G	W	g	w
1 0 0 0	8	BS	CAN	(	8	H	X	h	x
1 0 0 1	9	HT	EM	)	9	I	Y	i	y
1 0 1 0	10	LF	SUB	*	:	J	Z	j	z
1 0 1 1	11	VT	ESC	+	;	K	[	k	{
1 1 0 0	12	FF	FS	,	<	L	\	l	|
1 1 0 1	13	CR	GS	-	=	M	]	m	}
1 1 1 0	14	SO	RS	.	>	N	^	n	~
1 1 1 1	15	SI	US	/	?	O	_	o	DEL

[General Electric TermiNet 300, ~1971]

```
char c0 = 0x61;
char c1 = 'a';
printf("char c0:%c  char c1:%c\n", c0, c1);
```

```
> char c0:a  char c1:a
```


© T. Gemmeke, IDS, RWTH

```
printf("%d\t%d\n", fahr, celsius);
```


© T. Gemmeke, IDS, RWTH

Einführendes C-Programmbeispiel

- Umwandlung von Grad Fahrenheit in Grad Celsius
- $\frac{y}{°C} = (5/9) \times \left(\frac{x}{°F} - 32\right)$

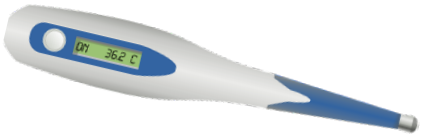
```
/* Umwandlung von Fahrenheit in Celsius
    fuer fahr = 0, 20, …, 300   */

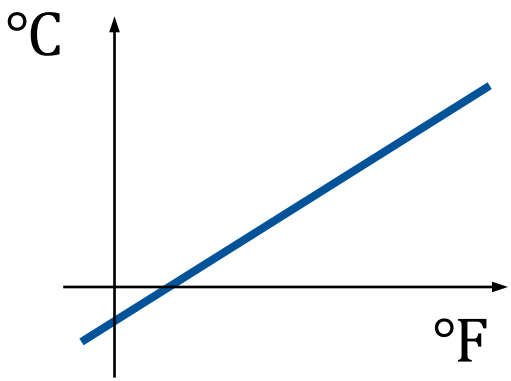

    int fahr, celsius;
    int lower

    lower = 0; // untere Grenze der Temperaturtabelle



    fahr  = lower;
    while (fahr <= upper)  {
        celsius = 5  *  (fahr-32) / 9;
        printf("%d\t%d\n", fahr, celsius);
        fahr = fahr + step;
    }
```





© T. Gemmeke, IDS, RWTH

Einführendes C-Programmbeispiel

- Umwandlung von Grad Fahrenheit in Grad Celsius
- $\frac{y}{°C} = (5/9) \times \left(\frac{x}{°F} - 32\right)$

```c
#include <stdio.h>

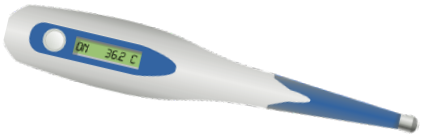
/* Umwandlung von Fahrenheit in Celsius
    fuer fahr = 0, 20, …, 300   */

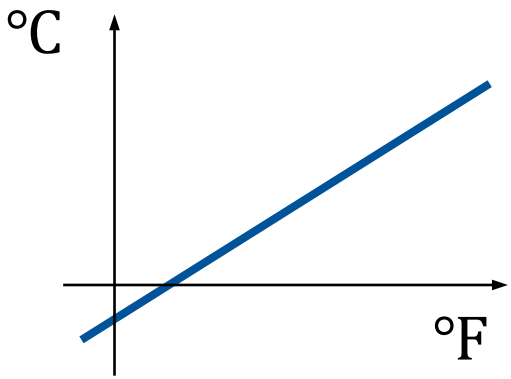

    int fahr, celsius;
    int lower

    lower = 0; // untere Grenze der Temperaturtabelle



    fahr  = lower;
    while (fahr <= upper)  {
        celsius = 5  *  (fahr-32) / 9;
        printf("%d\t%d\n", fahr, celsius);
        fahr = fahr + step;
    }
```





© T. Gemmeke, IDS, RWTH

Einführendes C-Programmbeispiel

- Umwandlung von Grad Fahrenheit in Grad Celsius
- $\frac{y}{°C} = (5/9) \times \left(\frac{x}{°F} - 32\right)$

```c
#include <stdio.h>

/* Umwandlung von Fahrenheit in Celsius
    fuer fahr = 0, 20, …, 300   */
int main()
{
    int fahr, celsius;
    int lower

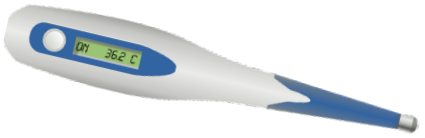
    lower = 0; // untere Grenze der Temperaturtabelle

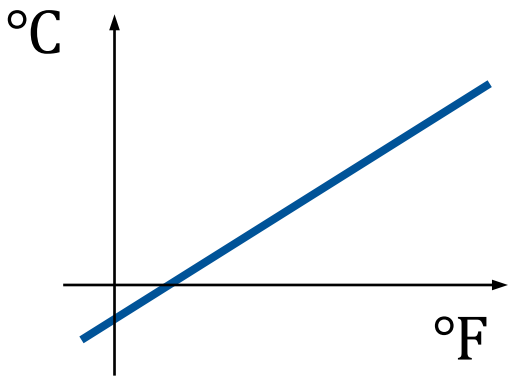


    fahr  = lower;
    while (fahr <= upper)  {
        celsius = 5  *  (fahr-32) / 9;

        fahr = fahr + step;
    }

    return 0;
}
```





© T. Gemmeke, IDS, RWTH

Einführendes C-Programmbeispiel

- Umwandlung von Grad Fahrenheit in Grad Celsius
- $\frac{y}{°C} = (5/9) \times \left(\frac{x}{°F} - 32\right)$

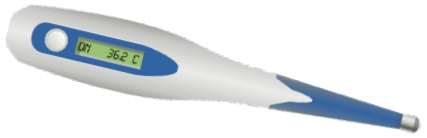
```c
#include <stdio.h>

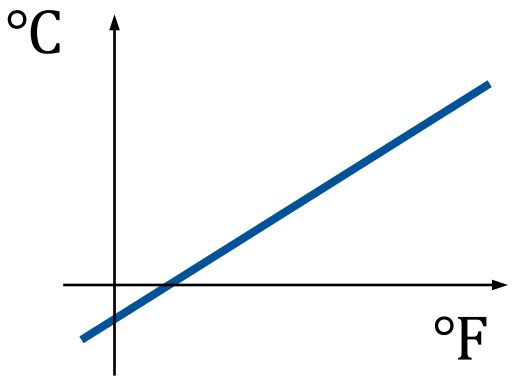
/* Umwandlung von Fahrenheit in Celsius
    fuer fahr = 0, 20, …, 300   */
int main()
{
    int fahr, celsius;
    int lower, upper, step;

    lower = 0; // untere Grenze der Temperaturtabelle
    upper = 300; // obere Grenze
    step  = 20; // Schrittweite

    fahr  = lower;
    while (fahr <= upper)  {
        celsius = 5  *  (fahr-32) / 9;
        printf("%d\t%d\n", fahr, celsius);
        fahr = fahr + step;
    }

    return 0;
}
```





© T. Gemmeke, IDS, RWTH

2. Einführendes Beispiel

2.1 Temperaturumrechnung

2.2 Analyse des Beispiels

2.3 Ausführung und Fehlersuche

Analyse des C-Programms

- **Einbinden** der Standard-Library zur Ein/Ausgabe
  - `#include <stdio.h>`
  - Ermöglicht Kommunikation des Programms mit der Außenwelt (z.B. Benutzer)
  - Siehe C-Präprozessor
- **Kommentare**
  - `/* Umwandlung … */`
  - `// untere Grenze …`
  - Erläuterung der Programmfunktion für Entwickler
  - Text innerhalb `/* … */` und in einer Zeile nach `//` wird vom Compiler ignoriert
- **Hauptfunktion** `main`
  - `int main() { … }`
  - Alle Anweisungen innerhalb `{ … }` gehören zu `main`
  - `main` wird nach Programmstart immer direkt aufgerufen
  - Im Allg. ruft `main` dann weitere Funktionen auf

© T. Gemmeke, IDS, RWTH

Analyse des C-Programms

- **Variablen-Vereinbarungen**
  - `int fahr, celsius;`
  - Variablen sind benannte Speicherplätze für Werte
  - Können lesend und schreibend zugegriffen werden
  - Variablen müssen vor Verwendung vereinbart werden
  - Hier: Verwendung von ganzzahligen (integer) Variablen
- **Zuweisungen**
  - `lower = 0;`
  - `upper = 300;`
  - Schreiben von Werten in die Variablen
- **Schleife**
  - `while (fahr <= upper) { /* ... */ }`
  - `while` ist (wie auch `int`) ein **C-Schlüsselwort**
  - Code innerhalb `{ … }` wird solange wiederholt, bis Bedingung nicht mehr erfüllt

© T. Gemmeke, IDS, RWTH

Analyse des C-Programms

- **Umrechnung**
  - `celsius = 5 * (fahr-32) / 9;`
  - Rechenoperationen auf Variablen
- **Ausgabe**
  - `printf("%d\t%d\n", fahr, celsius);`
  - Formatierte Ausgabe:
    - `fahr` als Integer-Zahl (erstes `%d` als Platzhalter)
    - Tabulator `\t`
    - `celsius` als Integer-Zahl (zweites `%d` als Platzhalter)
    - Zeilenumbruch `\n`
- Vorbereitung des **nächsten Schleifendurchlaufs**
  - `fahr = fahr + step;`
  - `fahr` wird um `step` erhöht, bis `upper` überschritten

© T. Gemmeke, IDS, RWTH

C-Präprozessor

- C-Präprozessor (cpp) läuft **vor dem Compiler** über den Quellcode (C-nach-C-Übersetzung)
- cpp-Anweisungen beginnen mit Zeichen #
- cpp-Verwendung erhöht **Lesbarkeit** der Programme
- Vor allem folgende Anwendungen:
  - **Symbolische Definitionen** (Konvention: Großbuchstaben)
    - `#define UPPER 300`
    - Alle Vorkommen von `UPPER` werden durch `300` ersetzt
  - **Makros**
    - `#define MAX(a,b) (a > b) ? a : b`
    - Alle Vorkommen von `MAX(..., ...)` werden ersetzt
  - **Einbinden** von Standard-Libraries (und allgemein Header-Dateien mit Deklarationen)
    - `#include <stdio.h>`
    - Datei `stdio.h` wird an dieser Stelle ins Programm eingefügt

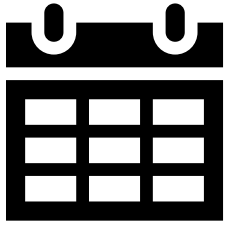
© T. Gemmeke, IDS, RWTH

© T. Gemmeke, IDS, RWTH

Zahlendarstellung

1.1.2022

$(\mathbf{2201010001})_{10}$



32.12.2021

$(\mathbf{2112320001})_{10}$

YY MM DD ####

$(\mathbf{22\ 01\ 01\ 0001})_{10}$

$(\mathbf{10000011001100001011111101010001})_2$

$(\mathbf{1000\ 0011\ 0011\ 0000\ 1011\ 1111\ 0101\ 0001})_2 = -(\mathbf{2093957295})_{10}$

$(\mathbf{83\ 30\ BF\ 51})_{16}$

Vorzeichen?

→ MSB (Most Significant Bit) hat Gewicht -2^{N-1}

hier: $-2^{31} = -(2147483648)_{10}$

Größte vorzeichenbehaftete 32b-Zahl:

$2^{N-1} - 1 = 2^{31-1} = (2147483647)_{10}$

© T. Gemmeke, IDS, RWTH